

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 - SORTING



MUHAMAD MIRZA AGUSTA

109082500044

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Praktikum Modul 14 ini berfokus pada konsep algoritma pengurutan (sorting) dalam pemrograman menggunakan bahasa Go. Sorting adalah proses mengatur sekumpulan data ke dalam urutan tertentu, baik ascending (terkecil ke terbesar) maupun descending (terbesar ke terkecil). Dua algoritma sorting yang dipelajari dalam praktikum ini adalah Selection Sort dan Insertion Sort, masing-masing dengan karakteristik, kelebihan, dan kekurangan tersendiri.

1. Selection Sort

Selection sort adalah algoritma pengurutan yang bekerja dengan mencari nilai ekstrem (minimum untuk ascending atau maksimum untuk descending) dari bagian array yang belum terurut pada setiap iterasi, kemudian menukarnya dengan elemen pertama dari bagian tersebut. Algoritma ini menggunakan dua perulangan yaitu perulangan luar untuk iterasi sorting dan perulangan dalam untuk mencari nilai ekstrem. Kompleksitas waktu selection sort adalah $O(n^2)$ dalam kasus terburuk maupun terbaik, karena algoritma ini tetap melakukan perbandingan sebanyak $n(n-1)/2$ kali meskipun data sudah terurut. Kelebihan selection sort adalah kesederhanaannya dan jumlah pertukaran yang minimal (hanya $n-1$ kali), sehingga cocok untuk data dengan ukuran kecil atau ketika operasi pertukaran lebih mahal daripada operasi perbandingan.

2. Insertion Sort

Insertion sort adalah algoritma pengurutan yang bekerja dengan membagi array menjadi dua bagian yaitu bagian yang sudah terurut (di sebelah kiri) dan bagian yang belum terurut (di sebelah kanan). Algoritma ini mengambil satu elemen dari bagian belum terurut (disebut key) dan menyisipkannya ke posisi yang tepat pada bagian terurut dengan cara menggeser elemen-elemen yang lebih besar (untuk ascending) atau lebih kecil (untuk descending) ke kanan. Kompleksitas waktu insertion sort adalah $O(n^2)$ dalam kasus terburuk, namun $O(n)$ dalam kasus terbaik ketika data sudah terurut. Kelebihan insertion sort adalah efisiensinya pada data yang hampir terurut, stabil (mempertahankan urutan relatif elemen yang sama), dan mudah diimplementasikan.

3. Perbandingan Selection Sort dan Insertion Sort

Selection sort dan insertion sort memiliki kompleksitas waktu yang sama yaitu $O(n^2)$, namun memiliki karakteristik berbeda. Selection sort melakukan jumlah perbandingan yang tetap ($n(n-1)/2$) terlepas dari kondisi data, sementara insertion sort lebih adaptif karena perbandingan berhenti lebih awal ketika key sudah berada di posisi yang tepat. Insertion sort juga bersifat stable (mempertahankan urutan relatif elemen dengan nilai yang sama), sedangkan selection sort bersifat unstable. Untuk data yang sudah hampir terurut, insertion sort jauh lebih efisien dibandingkan selection sort.

4. Sorting pada Tipe Data Berbeda

Algoritma sorting dapat diterapkan pada berbagai tipe data, tidak terbatas pada bilangan bulat. Pada array string, perbandingan dilakukan secara leksikografis (berdasarkan urutan abjad) menggunakan operator $<$ untuk ascending atau $>$ untuk descending. Go membandingkan string berdasarkan nilai ASCII/Unicode, sehingga bersifat case-sensitive. Sorting pada struct dapat dilakukan dengan membandingkan salah satu field

yang dijadikan sebagai kunci pengurutan (key), misalnya mengurutkan data buku berdasarkan field rating.

5. Pointer dalam Fungsi Sorting

Dalam bahasa Go, ketika array berukuran tetap (statis) dikirim ke fungsi, secara default array tersebut dikirim sebagai pass by value (salinan), sehingga perubahan tidak mempengaruhi array asli. Untuk memodifikasi array asli, diperlukan penggunaan pointer dengan sintaks `*[n]T` pada parameter fungsi, dan pengiriman alamat array menggunakan operator `&` saat pemanggilan fungsi. Pada slice, karena slice referensi ke array dasar, pengiriman slice ke fungsi tanpa pointer sudah cukup untuk memodifikasi data asli.

6. Pengurutan Multi-Daerah dan Pemisahan Data

Dalam aplikasi nyata, seringkali diperlukan pengurutan data yang berasal dari beberapa daerah atau kategori secara terpisah. Program dapat membaca jumlah daerah terlebih dahulu, kemudian melakukan perulangan untuk memproses setiap daerah. Selain itu, data juga dapat dipisahkan berdasarkan kondisi tertentu (misalnya memisahkan bilangan ganjil dan genap) sebelum diurutkan dengan algoritma yang sesuai (ascending untuk ganjil, descending untuk genap).

B. Guided

1. [Program Pengurutan Array dengan Algoritma Selection Sort]

```
package main

import "fmt"

func selecetionSortArray(angka *[5]int) {
    var idx_min, i, j int
    for i = 0; i < len(angka)-1; i++ { //loop luar (iterasi
        sortingnya)
            idx_min = i
            for j = i + 1; j < len(angka); j++ { //loop dalam
                (mencari nilai ekstrim)
                    if angka[j] <= angka[idx_min] { //sorting terkecil
                        ke terbesar
                            idx_min = j
                        }
                    }
            if idx_min != i {
                angka[i], angka[idx_min] = angka[idx_min], angka[i]
            }
        }
    }

func main() {
    var arrAngka [5]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka ke-%d : ", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()

    fmt.Println("=== SEBELUM DISORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }

    fmt.Println("\n=== SETELAH DISORITNG ===")
    selecetionSortArray(&arrAngka)
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }
}
```

Output :

```
082500044_MUHAMAD-MIRZA-AGUSTA\week 12\guided\n01\1.go"
Masukkan data angka ke-0 : 2
Masukkan data angka ke-1 : 4
Masukkan data angka ke-2 : 3
Masukkan data angka ke-3 : 7
Masukkan data angka ke-4 : 9

=== SEBELUM DISORTING ===
2 - 4 - 3 - 7 - 9 -
=== SETELAH DISORITNG ===
2 - 3 - 4 - 7 - 9 -
```

Penjelasan :

Program ini dideklarasikan dengan fungsi `selecetionSortArray` yang menerima pointer ke array berukuran 5 elemen bertipe integer, memungkinkan fungsi untuk memodifikasi langsung array asli tanpa mengembalikan nilai. Variabel `idx_min` digunakan untuk menyimpan indeks dari nilai minimum sementara, dan ketika ditemukan nilai yang lebih kecil, indeks tersebut diperbarui. Setelah perulangan dalam selesai, jika `idx_min` tidak sama dengan `i`, maka dilakukan pertukaran nilai antara `angka[i]` dan `angka[idx_min]` menggunakan teknik swapping yang elegan dalam Go. Dalam fungsi `main`, program meminta pengguna memasukkan 5 bilangan bulat, menampilkan array sebelum sorting, memanggil fungsi `selecetionSortArray` dengan mengirimkan pointer array menggunakan operator `&`, kemudian menampilkan array setelah sorting. Program ini merupakan contoh implementasi yang baik untuk memahami algoritma selection sort, penggunaan pointer dalam fungsi untuk memodifikasi array asli

2. [Program Selection Sort untuk Mengurutkan IPK Mahasiswa dari Terendah ke Tertinggi]

```
package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK        float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS)-1; i++ { //loop luar
        idx_min = i
```

```

        for j = i + 1; j < len(sMHS); j++ { //loop dalam
mencari nilai ekstrim

                if sMHS[j].IPK < sMHS[idx_min].IPK { //sorting
terkecil ke terbesar

                        idx_min = j

                }

        }

        if idx_min != i {

                sMHS[i], sMHS[idx_min] = sMHS[idx_min],
sMHS[i]

        }

}

func main() {

        var structMHS [5]mahasiswa

        for i := 0; i < len(structMHS); i++ {

                fmt.Printf("--- Masukkan Data Mahasiswa Ke-%d ---
\n", i)

                fmt.Print("Nama : ")

                fmt.Scan(&structMHS[i].nama)

                fmt.Print("NIM : ")

                fmt.Scan(&structMHS[i].nim)

                fmt.Print("IPK : ")

                fmt.Scan(&structMHS[i].IPK)

        }

```

```

        fmt.Println()

        fmt.Println(" === SEBELUM SORITNG === ")

        for i := 0; i < len(structMHS); i++ {

            fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)

            fmt.Printf("Nama : %s\n", structMHS[i].nama)

            fmt.Printf("NIM : %s\n", structMHS[i].nim)

            fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)

        }

        fmt.Println("=====")

        fmt.Println()

        SelectionSortArray(&structMHS)

        fmt.Println(" === SETELAH SORITNG === ")

        for i := 0; i < len(structMHS); i++ {

            fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)

            fmt.Printf("Nama : %s\n", structMHS[i].nama)

            fmt.Printf("NIM : %s\n", structMHS[i].nim)

            fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)

        }

        fmt.Println("=====")

        fmt.Println()

    }

    insertionSortString(&data, n)

```

```
    for i := 0; i < n; i++ {  
        fmt.Print(data[i], " ")  
    }  
}
```

Output :

```
week 12\guided\2.no2\2.go:1:1: expected 'package', found 'EOF'  
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run  
S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\week 12\guided\2.no2\2.g  
--- Masukkan Data Mahasiswa Ke-0 ---  
Nama : mirza  
NIM : 109082500044  
IPK : 3.67  
--- Masukkan Data Mahasiswa Ke-1 ---  
Nama : bruce  
NIM : 109082500042  
IPK : 3.99  
--- Masukkan Data Mahasiswa Ke-2 ---  
Nama : clark  
NIM : 109082500321  
IPK : 3.70  
--- Masukkan Data Mahasiswa Ke-3 ---  
Nama : tony  
NIM : 109082500023  
IPK : 3.89  
--- Masukkan Data Mahasiswa Ke-4 ---  
Nama : bima  
NIM : 109082500035  
IPK : 3.91
```



```

=== SETELAH SORTING ===
--- Data Mahasiswa Ke-0 ---
Nama : mirza
NIM : 109082500044
IPK : 3.67
--- Data Mahasiswa Ke-1 ---
Nama : clark
NIM : 109082500321
IPK : 3.70
--- Data Mahasiswa Ke-2 ---
Nama : tony
NIM : 109082500023
IPK : 3.89
--- Data Mahasiswa Ke-3 ---
Nama : bima
NIM : 109082500035
IPK : 3.91
--- Data Mahasiswa Ke-4 ---
Nama : bruce
NIM : 109082500042
IPK : 3.99
=====

```

Penjelasan :

Program dimulai dengan mendefinisikan struct `mahasiswa` yang memiliki tiga field yaitu `nama` (string), `nim` (string), dan `IPK` (float64), kemudian mendeklarasikan array `structMHS` berukuran 5 elemen untuk menyimpan data lima orang mahasiswa. Pengguna diminta memasukkan nama, NIM, dan IPK untuk setiap mahasiswa secara berurutan melalui perulangan, lalu program menampilkan data tersebut sebelum proses sorting dimulai. Fungsi `SelectionSortArray` menerima pointer ke array struct sebagai parameter, yang memungkinkan fungsi ini memodifikasi langsung array asli di fungsi `main` tanpa perlu mengembalikan nilai. Algoritma selection sort bekerja dengan melakukan perulangan luar dari indeks 0 hingga 3, di mana pada setiap iterasi, program mencari indeks dengan nilai IPK terkecil dari bagian array yang belum terurut (dimulai dari indeks i hingga akhir). Jika ditemukan IPK yang lebih kecil, indeks tersebut disimpan sebagai `idx\_min`. Setelah perulangan dalam selesai, program menukar posisi elemen ke-i dengan elemen ke-idx_min menggunakan teknik swapping `a, b = b, a` yang merupakan fitur elegan dalam Go. Proses ini terus berulang hingga seluruh array terurut secara ascending (dari IPK terendah ke tertinggi). Setelah sorting selesai, program menampilkan kembali data mahasiswa yang sudah terurut berdasarkan IPK. Program ini merupakan contoh implementasi yang sangat baik untuk memahami bagaimana algoritma selection sort dapat diterapkan pada data terstruktur (array of struct).

3. [Program Pengurutan Descending dengan Algoritma Insertion Sort]

```
package main
```

```

import "fmt"

const nMax int = 4321

type arrInt [nMax]int

func insertionSort(T *arrInt, n int) {
    /* I.S. terdefinisi array T yang berisi n bilangan
    bulat

    F.S. array T terurut secara mengecil atau
    descending dengan INSERTION SORT */

    var temp, i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]

        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}

```

```

    }
}

func main() {
    var data arrInt

    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

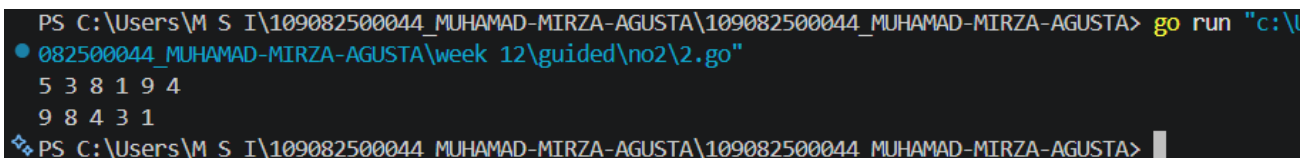
    insertionSort(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :

Penjelasan :



```

PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\week 12\guided\n02\2.go"
5 3 8 1 9 4
9 8 4 3 1
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA>

```

Program ini adalah implementasi algoritma insertion sort untuk mengurutkan array bilangan bulat secara descending (dari terbesar ke terkecil). Algoritma insertion sort bekerja dengan membagi array menjadi dua bagian yaitu bagian yang sudah terurut

dan bagian yang belum terurut, kemudian mengambil satu elemen dari bagian belum terurut dan menyisipkannya ke posisi yang tepat pada bagian terurut. Fungsi ini menggunakan variabel *i* sebagai penanda batas antara bagian terurut dan belum terurut, *j* untuk iterasi mundur saat mencari posisi yang tepat, serta *temp* untuk menyimpan nilai sementara yang akan disisipkan. Perulangan luar `for i <= n-1` berjalan dari indeks 1 hingga *n-1*, di mana pada setiap iterasi nilai *T[i]* disimpan ke *temp*. Perulangan dalam `for j > 0 && temp > T[j-1]` akan menggeser elemen ke kanan selama nilai *temp* lebih besar dari elemen sebelumnya, menciptakan ruang untuk penyisipan. Setelah perulangan dalam selesai, nilai *temp* disisipkan ke posisi *j* yang telah ditemukan. Dalam fungsi `main`, program membaca jumlah data *n* dan *n* buah bilangan bulat, memanggil fungsi `insertionSort` dengan mengirimkan pointer array menggunakan operator `&`, kemudian menampilkan array yang sudah terurut secara descending. Program ini merupakan implementasi yang baik untuk memahami algoritma insertion sort.

4. [Program Pengurutan String Ascending dengan Algoritma Insertion Sort]

```
package main

import "fmt"

const nMax int = 100

type arrString [nMax]string

func insertionSortString(T *arrString, n int) {
    var temp string
    var i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]
```

```

        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}

```

```

func main() {
    var data arrString
    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    insertionSortString(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

```
}
```

Output :

```
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run 082500044_MUHAMAD-MIRZA-AGUSTA\week 12\guided\no3\3.go"
4
mangga
apel
jeruk
pisang
apel jeruk mangga pisang
```

Penjelasan :

Program ini adalah implementasi algoritma insertion sort untuk mengurutkan array string secara ascending menggunakan. Fungsi insertionSortString menerima pointer ke array string dan jumlah elemen, lalu mengurutkan data dengan membandingkan string menggunakan operator < untuk menentukan urutan abjad. Perulangan luar berjalan dari indeks 1 hingga n-1, menyimpan nilai sementara ke variabel temp, kemudian perulangan dalam menggeser elemen ke kanan selama temp lebih kecil dari elemen sebelumnya. Setelah posisi tepat ditemukan, nilai temp disisipkan. Dalam fungsi main, program membaca jumlah data dan elemen string, memanggil fungsi sorting, lalu menampilkan hasil urutan dari terkecil ke terbesar. Program ini menunjukkan bahwa algoritma sorting dapat diterapkan pada berbagai tipe data dan mendemonstrasikan penggunaan pointer serta perulangan bersarang.

C. Unguided

1. Soal Unguided (selectionsort)

Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda insertion sort), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Jawaban :

```
package main

import (
    "fmt"
)

Selection Sort (Ascending)
func selectionSort(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        minIndex := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIndex] {
                minIndex = j
            }
        }

        if minIndex != i {
            arr[i], arr[minIndex] = arr[minIndex], arr[i]
        }
    }
}

func main() {
    var n int

    _, err := fmt.Scan(&n)
    if err != nil {
        return
    }

    for i := 0; i < n; i++ {
        var m int

        fmt.Scan(&m)

        rumah := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&rumah[j])
        }
    }
}
```

```

        selectionSort(rumah)

        for k := 0; k < m; k++ {
            if k > 0 {
                fmt.Print(" ")
            }
            fmt.Print(rumah[k])
        }
        fmt.Println()
    }
}

```

Output :

```

PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\week 12\unguided\no1\1.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9

```

Penjelasan :

Program ini adalah aplikasi untuk mengurutkan nomor rumah di beberapa daerah menggunakan algoritma selection sort. Program dimulai dengan membaca bilangan bulat *n* yang menunjukkan jumlah daerah yang akan diproses, kemudian melakukan perulangan sebanyak *n* kali. Untuk setiap daerah, program membaca bilangan *m* sebagai jumlah rumah, kemudian membaca *m* buah nomor rumah ke dalam slice *rumah*. Fungsi *selectionSort* digunakan untuk mengurutkan nomor rumah secara ascending dengan algoritma selection sort yang bekerja dengan mencari nilai minimum dari bagian array yang belum terurut pada setiap iterasi, lalu menukarnya dengan elemen pertama dari bagian tersebut. Setelah terurut, program menampilkan nomor rumah dalam satu baris dengan spasi sebagai pemisah, kemudian melanjutkan ke daerah berikutnya. Program ini merupakan implementasi yang baik untuk memahami algoritma selection sort, pembuatan slice dinamis, serta pengolahan data multi-daerah.

2. Soal Unguided

Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka

selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisikiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program kerabat dekat yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Jawaban :

```
package main

import (
    "fmt"
)

func selectionSortAsc(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        minIdx := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIdx] {
                minIdx = j
            }
        }
        arr[i], arr[minIdx] = arr[minIdx], arr[i]
    }
}

func selectionSortDesc(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        maxIdx := i
        for j := i + 1; j < n; j++ {
            if arr[j] > arr[maxIdx] {
                maxIdx = j
            }
        }
        arr[i], arr[maxIdx] = arr[maxIdx], arr[i]
    }
}

func main() {
    var n int
    fmt.Scan(&n)

    for daerah := 0; daerah < n; daerah++ {
        var m int
        fmt.Scan(&m)

        rumah := make([]int, m)
        for i := 0; i < m; i++ {
```

```

        fmt.Scan(&rumah[i])
    }

    var ganjil, genap []int
    for _, num := range rumah {
        if num%2 == 1 {
            ganjil = append(ganjil, num)
        } else {
            genap = append(genap, num)
        }
    }

    selectionSortAsc(ganjil)
    selectionSortDesc(genap)

    for i, v := range ganjil {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(v)
    }
    for i, v := range genap {
        if len(ganjil) > 0 || i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(v)
    }
    fmt.Println()
}

```

Output :

```

PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA>
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4

```

Penjelasan :

Program Program ini adalah aplikasi untuk mengurutkan nomor rumah di beberapa daerah dengan memisahkan bilangan ganjil dan genap menggunakan algoritma selection sort. Program dimulai dengan membaca bilangan bulat n yang menunjukkan jumlah daerah yang akan diproses, kemudian melakukan perulangan sebanyak n kali. Untuk setiap daerah, program membaca bilangan m sebagai jumlah rumah, lalu membaca m buah nomor rumah ke dalam slice rumah. Selanjutnya, program melakukan iterasi untuk memisahkan nomor rumah tersebut: nomor ganjil dimasukkan ke dalam slice ganjil dan nomor genap ke dalam slice genap. Fungsi selectionSortAsc digunakan untuk mengurutkan nomor rumah ganjil secara ascending (menaik), sedangkan fungsi selectionSortDesc digunakan untuk mengurutkan nomor rumah genap secara descending (menurun). Kedua fungsi ini bekerja dengan mencari nilai ekstrem (minimum atau maksimum) dari bagian array yang belum terurut pada setiap iterasi, lalu menukarnya dengan elemen pertama dari bagian tersebut. Setelah kedua slice terurut, program menampilkan seluruh nomor rumah ganjil terlebih dahulu baru diikuti nomor genap dalam satu baris dengan spasi sebagai pemisah yang rapi, kemudian melanjutkan ke daerah berikutnya. Program ini merupakan implementasi yang baik untuk memahami variasi algoritma selection sort, pemisahan data berbasis kondisi (paritas), pembuatan slice dinamis, serta pengolahan data multi-daerah.

1. Soal Unguided (insertionsort)

Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda insertion sort), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Jawaban :

```
package main

import (
    "fmt"
)

func insertionSort(arr []int) {
    n := len(arr)
    for i := 1; i < n; i++ {
        key := arr[i]
        j := i - 1

        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }
        arr[j+1] = key
    }
}

func main() {
    var arr []int

    for {
        var num int
        _, err := fmt.Scan(&num)
```

```

        if err != nil {
            break
        }

        if num < 0 {
            break
        }

        arr = append(arr, num)
    }

    if len(arr) == 0 {
        return
    }

    insertionSort(arr)

    for i, val := range arr {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(val)
    }
    fmt.Println()

    isConstant := true
    var delta int

    if len(arr) > 1 {

        delta = arr[1] - arr[0]
        for i := 1; i < len(arr)-1; i++ {
            if (arr[i+1] - arr[i]) != delta {
                isConstant = false
                break
            }
        }
    } else {

        delta = 0
    }

    if isConstant {
        fmt.Printf("Data berjarak %d\n", delta)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}

```

Output :

```
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\1\tempCodeRunnerFile.go"
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
```

Penjelasan :

Program ini adalah aplikasi untuk mengurutkan bilangan bulat menggunakan algoritma insertion sort, kemudian menganalisis apakah bilangan-bilangan tersebut membentuk barisan aritmatika (memiliki selisih yang tetap). Program membaca bilangan bulat secara terus menerus hingga ditemukan bilangan negatif, lalu menyimpannya ke dalam slice menggunakan append. Setelah itu, program mengurutkan data secara ascending dengan fungsi insertionSort yang bekerja dengan mengambil satu elemen (key) dan menyisipkannya ke posisi yang tepat pada bagian array yang sudah terurut. Data terurut kemudian ditampilkan dalam satu baris. Selanjutnya, program menganalisis selisih antar elemen dengan menghitung delta dari dua elemen pertama, lalu memeriksa apakah semua pasangan bilangan berurutan memiliki selisih yang sama. Jika ya, program menampilkan "Data berjarak X", jika tidak, menampilkan "Data berjarak tidak tetap".

2. Soal Unguided

Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan.

Jawaban :

```
package main

import "fmt"

const nMax int = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {

    fmt.Scan(n)
    for i := 0; i < *n; i++ {
        fmt.Scan(&pustaka[i].id)
        fmt.Scan(&pustaka[i].judul)
        fmt.Scan(&pustaka[i].penulis)
        fmt.Scan(&pustaka[i].penerbit)
        fmt.Scan(&pustaka[i].eksemplar)
        fmt.Scan(&pustaka[i].tahun)
        fmt.Scan(&pustaka[i].rating)
```

```

    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {

    if n == 0 {
        return
    }
    maxIdx := 0
    for i := 1; i < n; i++ {
        if pustaka[i].rating > pustaka[maxIdx].rating {
            maxIdx = i
        }
    }
    fmt.Printf("%s %s %s %d\n", pustaka[maxIdx].judul,
pustaka[maxIdx].penulis, pustaka[maxIdx].penerbit,
pustaka[maxIdx].tahun)
}

func UrutBuku(pustaka *DaftarBuku, n int) {

    for i := 1; i < n; i++ {
        key := pustaka[i]
        j := i - 1
        for j >= 0 && pustaka[j].rating < key.rating {
            pustaka[j+1] = pustaka[j]
            j--
        }
        pustaka[j+1] = key
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {

    limit := 5
    if n < limit {
        limit = n
    }
    for i := 0; i < limit; i++ {
        fmt.Println(pustaka[i].judul)
    }
}

func CariBuku(pustaka DaftarBuku, n int, r int) {

    left, right := 0, n-1
    found := -1
    for left <= right {
        mid := (left + right) / 2
        if pustaka[mid].rating == r {
            found = mid
            break
        } else if pustaka[mid].rating < r {
            right = mid - 1
        }
    }
}

```

```

        } else {
            left = mid + 1
        }
    }
    if found == -1 {
        fmt.Println("Tidak ada buku dengan rating seperti itu")
    } else {
        b := pustaka[found]
        fmt.Printf("%s %s %s %d %d %d\n", b.judul, b.penulis,
b.penerbit, b.tahun, b.eksemplar, b.rating)
    }
}

func main() {
    var pustaka DaftarBuku
    var nPustaka int

    DaftarkanBuku(&pustaka, &nPustaka)

    CetakTerfavorit(pustaka, nPustaka)

    UrutBuku(&pustaka, nPustaka)

    Cetak5Terbaru(pustaka, nPustaka)

    var cariRating int
    fmt.Scan(&cariRating)

    CariBuku(pustaka, nPustaka, cariRating)
}

```

Output :

```

PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\week 12\U2\tempCodeRunnerFile.go"
3
B01 Laskar-Pelangi Andrea-Hirata Bentang 10 2005 5
B02 Bumi-Manusia Pramoedya-Ananta-Toer Lentera-Dipantara 5 1980 4
B03 Negeri-5-Menara A.Fuadi Gramedia 15 2009 3
Laskar-Pelangi Andrea-Hirata Bentang 2005
Laskar-Pelangi
Bumi-Manusia
Negeri-5-Menara
4
Bumi-Manusia Pramoedya-Ananta-Toer Lentera-Dipantara 1980 5 4

```

Penjelasan :

Program ini data buku perpustakaan yang mencakup pencatatan, pengurutan, pencarian, dan tampilan informasi buku berdasarkan rating. Program mendefinisikan struct Buku dengan field id, judul, penulis, penerbit, eksemplar, tahun, dan rating, serta tipe Daftar Buku sebagai array statis dengan kapasitas maksimal 7919 elemen. Prosedur Daftarkan Buku membaca jumlah buku kemudian mengisi data setiap buku ke dalam array. Prosedur Cetak Terfavorit mencari buku dengan rating tertinggi menggunakan algoritma sequential search dan menampilkan judul, penulis, penerbit, serta tahun terbitnya. Prosedur UrutBuku mengurutkan buku secara descending berdasarkan rating menggunakan algoritma insertion sort, di mana buku dengan rating lebih besar akan bergeser ke kiri. Prosedur Cetak Terbaru menampilkan judul dari 5 buku pertama setelah diurutkan (yang memiliki rating tertinggi). Prosedur CariBuku melakukan pencarian berdasarkan rating menggunakan binary search, memanfaatkan data yang sudah terurut, dan menampilkan detail buku jika ditemukan atau pesan tidak ditemukan jika rating tidak ada. Dalam fungsi main, program memanggil prosedur secara berurutan untuk mendaftarkan buku, menampilkan buku terfavorit, mengurutkan buku, menampilkan 5 buku dengan rating tertinggi, membaca rating yang dicari dari input, dan mencari buku dengan rating tersebut.

D. Kesimpulan

Berdasarkan praktikum Modul 14 tentang sorting (pengurutan data) ini, dapat disimpulkan bahwa algoritma pengurutan merupakan fondasi penting dalam pengolahan data yang memungkinkan program untuk menyusun data menjadi urutan yang teratur, sehingga memudahkan proses pencarian dan analisis data selanjutnya. Melalui serangkaian program guided dan unguided, praktikum ini berhasil mendemonstrasikan dua algoritma sorting utama yaitu selection sort dan insertion sort, masing-masing dengan karakteristik implementasi yang berbeda. Pada guided pertama, program selection sort berhasil mengimplementasikan pengurutan ascending pada array 5 bilangan bulat menggunakan pointer agar array asli termodifikasi, dengan algoritma yang mencari nilai minimum dari bagian belum terurut lalu menukarnya ke posisi yang tepat. Guided kedua dan ketiga mengimplementasikan insertion sort untuk mengurutkan bilangan bulat secara descending dan string secara ascending, menunjukkan bahwa algoritma sorting dapat diadaptasi untuk berbagai tipe data dan arah pengurutan dengan hanya mengubah operator perbandingan.

Pada unguided pertama, program berhasil mengimplementasikan selection sort untuk mengurutkan nomor rumah di beberapa daerah, menunjukkan kemampuan program membaca jumlah daerah dan memproses data per daerah secara terpisah. Unguided kedua mengembangkan konsep pemisahan data di mana bilangan ganjil dipisahkan dari bilangan genap, kemudian ganjil diurutkan ascending menggunakan selection sort dan genap diurutkan descending, menampilkan hasil dalam satu baris dengan format yang rapi. Unguided ketiga mengimplementasikan insertion sort pada data integer yang dibaca secara dinamis hingga ditemukan bilangan negatif, kemudian menganalisis apakah data yang sudah terurut membentuk barisan aritmatika (selisih tetap) dengan memeriksa perbedaan setiap pasangan elemen berurutan.

Unguided keempat merupakan program yang paling kompleks, mengimplementasikan sistem manajemen perpustakaan lengkap dengan struct Buku, array statis berkapasitas 7919 buku, serta empat prosedur utama yaitu DaftarkanBuku untuk membaca data buku, CetakTerfavorit untuk mencari buku dengan rating tertinggi menggunakan sequential search, UrutBuku untuk mengurutkan buku secara descending berdasarkan rating menggunakan insertion sort, Cetak5Terbaru untuk menampilkan 5 buku dengan rating tertinggi, dan CariBuku untuk mencari buku berdasarkan rating menggunakan binary search yang efisien (karena data sudah terurut). Program ini menunjukkan integrasi antara sorting dan searching, di mana sorting dilakukan terlebih dahulu agar proses binary search dapat berjalan dengan benar. Secara keseluruhan, praktikum Modul 14 ini memberikan pemahaman yang mendalam tentang perbedaan karakteristik selection sort ($O(n^2)$ dengan jumlah perbandingan tetap) dan insertion sort ($O(n^2)$ dengan kasus terbaik $O(n)$), pentingnya pointer untuk memodifikasi array statis, adaptasi algoritma untuk berbagai tipe data dan arah pengurutan, pemisahan data berdasarkan kondisi, serta integrasi sorting dengan searching dalam sistem manajemen data yang kompleks. Pemahaman tentang algoritma sorting ini akan sangat berguna dalam pengembangan aplikasi yang membutuhkan penyajian data terurut, seperti sistem manajemen inventaris, basis data, aplikasi e-commerce, dan berbagai sistem informasi lainnya.